

specialised into standalone kernels and locked against any modification once deployed. In traditional operating systems like Linux, the virtual address space of any process is divided into two subspaces -- kernel space and user space. Unikernels do not have such defined boundaries as they are designed to run a single process that is tightly coupled with the kernel itself. The kernel and the application process are consolidated into an executable image that can run on a hypervisor, without the need for a separate operating system. The image contains the application code as well as all the operating system functions required by that application.

Unikernel operating systems can be compiled or built in different ways for a vast range of applications that run on different hardware platforms. Generally, unikernels can run in containers, on hypervisors, and on bare-metal hardware. Some unikernel systems are language-agnostic and can run any application code. Others, which can also be classified as virtual library operating systems, leverage high-level languages and runtime to provide an API for operating system functionality.

We can classify unikernels into two major kinds -- legacy and clean slate. The security profile of both varies slightly.

Legacy: Legacy unikernels implement a subset of POSIX to ensure unmodified software or programs can run, while some only require minor configuration changes.

They mostly focus on ensuring Linux compatibility and interoperability. Rumpun, OSv, and Nanos are some projects that support applications developed in different programming languages, and are designed to run a single unmodified Linux application.

Clean slate: Clean slate unikernels do not emulate classic operating systems. They are written in a single programming language with all OS functions like networking, memory

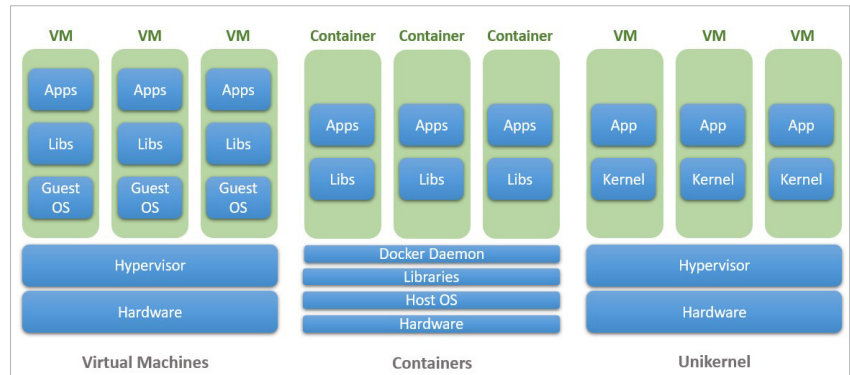


Figure 1: VMs vs containers vs unikernel architecture

management, and device drivers implemented in the same language. Clean slate unikernels allow language-specific virtual machines, like the Java virtual machine, to function as actual virtual machines. These are the most secure unikernels but are not capable of running programs or software developed for more traditional operating systems. A few popular unikernel projects in this category are Runtime.js, MirageOS, HaLVM, IncludeOS, and Clive.

Why unikernels?

Unikernels have a lot of advantages over traditional operating systems, the most important being improved security. They deploy single-purpose appliances. Although a VM contains many manageable layers of software, ultimately most VMs are utilised for a single purpose such as hosting a database or a web service. This reduces the amount of code deployed and services running on the instance, which in turn reduces the attack surface and results in improved security. Moreover, unikernel based infrastructure is immutable or read-only -- it employs the destroy and provision approach. Unikernels also do not allow us to SSH into them as there is no shell, no user accounts, and minimal libraries. Most importantly, they embrace the single process model which does not allow threat actors to easily inject their process or malware. Apart from

that, unikernels fit better within the DevOps pipeline because they add more flexibility and agility. Listed below are some of the advantages that reinforce their importance in cloud based infrastructures and DevOps.

Smaller footprint and portability:

Unikernel images are a lot smaller in size than traditional OS deployments. We can create and deploy submegabyte unikernels depending on our needs. Because of the smaller footprint, unikernels can run on a vast range of hardware where resources are a constraint. Being smaller in size also increases portability, and allows movement from one machine to another easily.

Highly optimised: Unikernels can perform better due to their single process nature and greater pairing with the kernel. A traditional OS contains more than a million lines of code, whereas a unikernel has only a few thousand lines of code to run a specific application.

Fast boot: Unikernels can boot extremely quickly, with boot times measured in milliseconds. On the cloud, they can be lightning fast which helps in creating a highly elastic infrastructure. Because unikernels are highly optimised and portable, they add to the scalability of any system. This also makes them ideal for DevOps, where on-demand provisioning of application instances with minimal delay can save costs and effort. For example, we can configure the pipeline